COMPUTER ENGINEERING
QUEZON CITY

Final Project: Barangay Flood Report and Clogged Drainage Tracking System

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

Item	Details
Project Title	Barangay Flood Report and Clogged Drainage Tracking System
Business Process Emulated	Barangay Flood Prioritization Complaint and Drainage Monitoring Maintenance System
Section / Group No.	CPE21S1 / Group 4
Instructor	Engr. Jimlord Quejado
Date Submitted	Oct 27, 2026
GitHub Repository	[https://github.com/...] (details in Appendix C)

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

General Objective: To develop a C++ program that emulates the flood reports and clogged drainage tracking process of barangay using appropriate data structures and algorithms.

Specific Objectives:

1. To manage incoming flood and clogged drainage reports in a barangay using a queue so that the reports are recorded and served in first-come, first-served order.
2. To search report records efficiently by report ID using binary search, and by street name using linear search
3. To sort flood reports by severity level and to organize reports by status which are pending or fixed, using merge sort, so the most cases are handled first and fixed problems can be tracked.
4. To analyze the time complexity of the main operations enqueue and dequeue, search, sort theoretically using big-O notation and experimentally by measuring runtime on different input sizes.

COMPUTER ENGINEERING
QUEZON CITY

1. BUSINESS PROCESS

1.1 Organization and Business Process

The organization is a barangay hall in Metro Manila (Damayang Lagi, Quezon City) that handles flood and drainage complaints. The people involved are the residents who report problems, the barangay staff who record it, the barangay captain or kagawad who decides what to act on first, and the clean-up crew who fix the problems or complaints.

The process starts when a resident reports a flooded street or clogged drain by call, text or walk in. The staff will put down the details,decide which report to handle first, and assign a crew. After the crew fixes the problem the staff will mark it as done. The output is the updated status of each report and a summary report for the barangay captain or barangay hall.

1.2 Current Process (As-Is)

Step	Activity	Performed By	Data Involved
1	Residents report a flood or blocked drain through text, call, or in person.	Barangay Resident	Name, Street Address and Issue Details
2	Staff writes down the report details manually on a logbook or stickynote.	Barangay Staff	Time, Date, Location, Flood Height and Place Condition
3	The officer looks through the logbook or the sticky note pages to see which street is the most affected.	Barangay Officer	Street Address and Decision Making
4	The barangay's flood team goes out to the designated street to clear drains or check the flooded area.	Barangay Flood Crew	Flood Crew Details and Assigned Street
5	Barangay Staff marks the logbook or removes the sticky note after the flood crew finishes the job.	Barangay Staff	Logboo0k's Rescue Notes and Flood Crew's Informations

1.3 Problems in the Current Process

1. The current process has manual and disorganized complaint recording by creating a complaint through text, personal or email complaint. This will be recorded to a logbook by a barangay staff which may lead to lost reports about the complaints created and difficulty to look for previous reports recorded.
2. The current process has difficulty determining which problem of clogged and flood reports need urgent attention because there is no organized system of ranking which tracks the severity of the problem and determines which problem has urgent priorities of attention.
3. The current process has difficulty assigning the barangay staff who are going to address the report about flooding and also clogged drainage problems because there is no tracking. This may cause confusion about what problems should be solved first and may delay the work if it is not assigned.

1.4 Proposed System

#	Feature	Business Task It Supports
1	Record the incoming complaint about the problems which are flooding and clogged drainage reports using the queue method.	Managing all incoming complaints which is FIFO(first come first served order).
2	Find a report using its report ID (binary search) or list all reports of a street using its street name (linear search)	To find what specific complaints reports need to be solved.
3	Sort the report by how the severity of the problem to determine what urgent action needs to be done by using merge sort.	Prioritize the area that has frequently reported flooding and clogged drainage.
4	Update and display the status of the complaint report to monitor it.	Monitoring the report progress and records what action has been done.
5	Count the total of report of the same area and its status	Determine how many complaints were made on the same problem.

1.5 Scope and Delimitations

- The system covers recording barangay flood and clogged drainage reports, such as the report ID, street or block, flood level, date and time, and the status of the report.
- The system covers handling the reports using a queue so that the reports are served first-come first,served.
- The system covers searching reports by report ID using binary search and by street/block name using linear search .
- The system covers sorting of the reports by its severity using merge sort so the serious problems can be seen first.
- The system does not have a graphical interface, database or internet. It is a console program made in C++ and the data is only saved in text files.
- The system is for one user only, like in a barangay staff member. The complaints are received through phone calls, text, or personal walk-ins and the staff will type the reports manually into the program.
- The system does not use maps, GPS or weather data, and it does not assign crews or send notifications. The flood level is only entered by the user.
- The system does not predict future floods or find the cause of the flooding. It only records and tracks the reports that the staff entered.

2. DATA STRUCTURES AND ALGORITHMS APPLIED

Course topics you may choose from: arrays and dynamic memory allocation; singly, doubly, and circular linked lists; stacks (array or linked-list based); queues (array or linked-list based); linear search; binary search; hashing; sorting algorithms; general trees and binary search trees; AVL trees; tree traversal algorithms; graphs (adjacency matrix or adjacency list); breadth-first search; depth-first search.

2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	Queue (linked list)	Pending reports waiting for a crew, one queue per severity	enqueue, dequeue, peek, isEmpty
2	Linear Search	Finding all reports of one street	Search by street name
3	Merge Sort	Ranking unresolved reports by severity and ranking streets by flood count	merge sort with capacitor
4	Stack (linked list)	Undo history of reports that marked as fixed	push, pop, peek
5	Binary Search	Finding a report by its report number	search by ID
6	Dynamic Array	Archive of all reports, kept in order of report ID	Push_back (with resizing), index access

2.2 Queue / Linked List

Concept: A queue works like a real-life line of people waiting. For Example The first person to get in line is the first person served FIFO. A linked list connects each item using links, so new items can be added without running out of fixed space.

Application in the System: When barangay residents reports a flood or clogged drain, the report goes into the back of the queue. Barangay Staff work on reports starting from the front of the line so complaints are handled in the exact order they arrived.

Justification: A queue makes sure older complaints are not forgotten when new ones come in. A linked list is a better option because the barangay staff does not know how many reports will arrive during a heavy storm.

Operations and Complexity :

- **Enqueue** (Add Report): **O(1)**
- **Dequeue** (Remove Finished Task): **O(1)**
- The main queue operations take **O(1)** time per action.
- It needs **O(n)** total space to store n reports in memory.

2.3 Merge Sort

Concept: Merge sort uses divide and conquer and it splits the list into halves again and again until each part has only one item then it merges back together in sorted order. For example if the reports have severity levels 2, 4, 1, and 3, merge sort splits them into single reports then merges them back by comparing, so the final list becomes 4, 3, 2, 1 with the most serious report first.

Application in the System: Merge sort arranges the flood reports by severity so the serious ones are seen first. It uses the report records and the flood level is what is compared.

1. The staff chooses the sort option.
2. The program splits the reports into halves until one report is left in each part.
3. The program merges them back, putting the higher flood level first.

4. The program shows the sorted list.

Justification: Merge sort is better than bubble or insertion sort because those take O(n^2) time and get slow when there are many reports like during a typhoon. Merge sort is always O(n log n) and is stable so reports with the same severity keep their order.

Operations and Complexity:

- Splitting the list into halves takes **O(log n)** level.
- Merging the sorted parts takes **O(n)** for each level.
- The whole merge sort takes **O(n log n)** time.
- It needs **O(n)** extra space for the merging.

2.4 Stack

Concept: A stack follows last in, first out (LIFO). The last item added is the first one removed and the main operations are push, pop and peek. For example, if the staff marks reports 101, 102 and 103 as fixed then presses undo, 103 is removed first because it was the last one added.

Application in the System: The stack keeps the reports that were recently marked as fixed so the staff can undo the last one if it was a mistake.

1. The staff chooses the mark as a fixed option and types the report ID.
2. The program changes the status of the report.
3. The program pushes the report onto the stack
4. If there is a mistake, the staff chooses to undo.
5. The program pops the top report and changes its status back to pending.

Justification: A stack is better than a queue for this because undo needs the most recent reports and a stack gives it right away. A queue would give the oldest one which is wrong. The program only works on the top, so it does not need to search the list.

Operations and Complexity:

- Push takes **O(1)**
- Pop takes **O(1)**
- Peek takes **O(1)**
- Checking if the stack is empty takes **O(1)**

2.5 Binary Search

Concept: Binary Search Finds an item in a sorted list by checking the middle item first. If the middle item is not the one that needs to be found, it throws away the half that does not contain it and keeps cutting the remaining list in half until it finds the match.

Application in the System: When barangay staff need to find a specific report using its Report ID , Binary Search locates it easily.

Justification: Binary Search is much faster than checking records one by one from top to bottom. For example, with 1,000 reports, checking one by one can take up to 1,000 steps, but binary search only needs about 10 steps. The reports must be sorted by Report ID first, and they already are because new reports get the next ID number

Operations and Complexity:

- Search by Report ID: best case **O(1)** (the middle item is the match), average and worst case **O(log n)**
- Each step cuts the list in half, so 1,000 reports take about 10 steps and 100,000 take about 17.
- It needs **O(1)** extra space.

2.6 Linear Search

Concept: Linear search checks the items one by one from the start of the list until it finds a match or reaches the end. It doesn't need the list to be sorted.

Application in the System: When the staff wants to see the reports of one street, the program searches by street name.

1. The staff types the street name.
2. The program checks the first report, then the next, and so on.
3. Every report with a matching street is shown.
4. If none match, the program shows a "no report found" message.

Justification: Binary search stops at the first match, but one street can have many reports. Linear search checks every record, so it finds all of them, and the list doesn't need to be sorted first. It's simple, and it's fast enough for a barangay-sized list.

Operations and Complexity:

- Search by street name: best case **O(1)** (first item matches), average and worst case **O(n)**
- It needs **O(1)** extra space.

2.7 Dynamic Array

Concept: A dynamic array works like a normal array, but it can grow. When it gets full, the program makes a bigger array (double the size), copies the old items into it, and deletes the old one. Any item can be reached right away using its index number.

Application in the System: The dynamic array stores the archive of all flood reports in order of report ID.

1. The staff adds a new report.
2. The program puts it at the end of the array.
3. If the array is full, the program doubles its size first.
4. Binary search, linear search and merge sort all read the reports from this array.

Justification: A normal array has a fixed size, and the barangay can't know how many reports will come in during a typhoon. A dynamic array grows when needed. It is better than a linked list here because binary search needs to jump straight to the middle item, and a linked list can't do that. New report IDs always go up by one, so the array stays sorted by ID without extra work.

Operations and Complexity:

- Index access: **O(1)**
- Add at the end (push_back): **O(1)** on average. A resize costs **O(n)**, but it only happens when the array is full.
- It needs **O(n)** space to store n reports.

3. SYSTEM FLOWCHART

3.1 Main System Flowchart

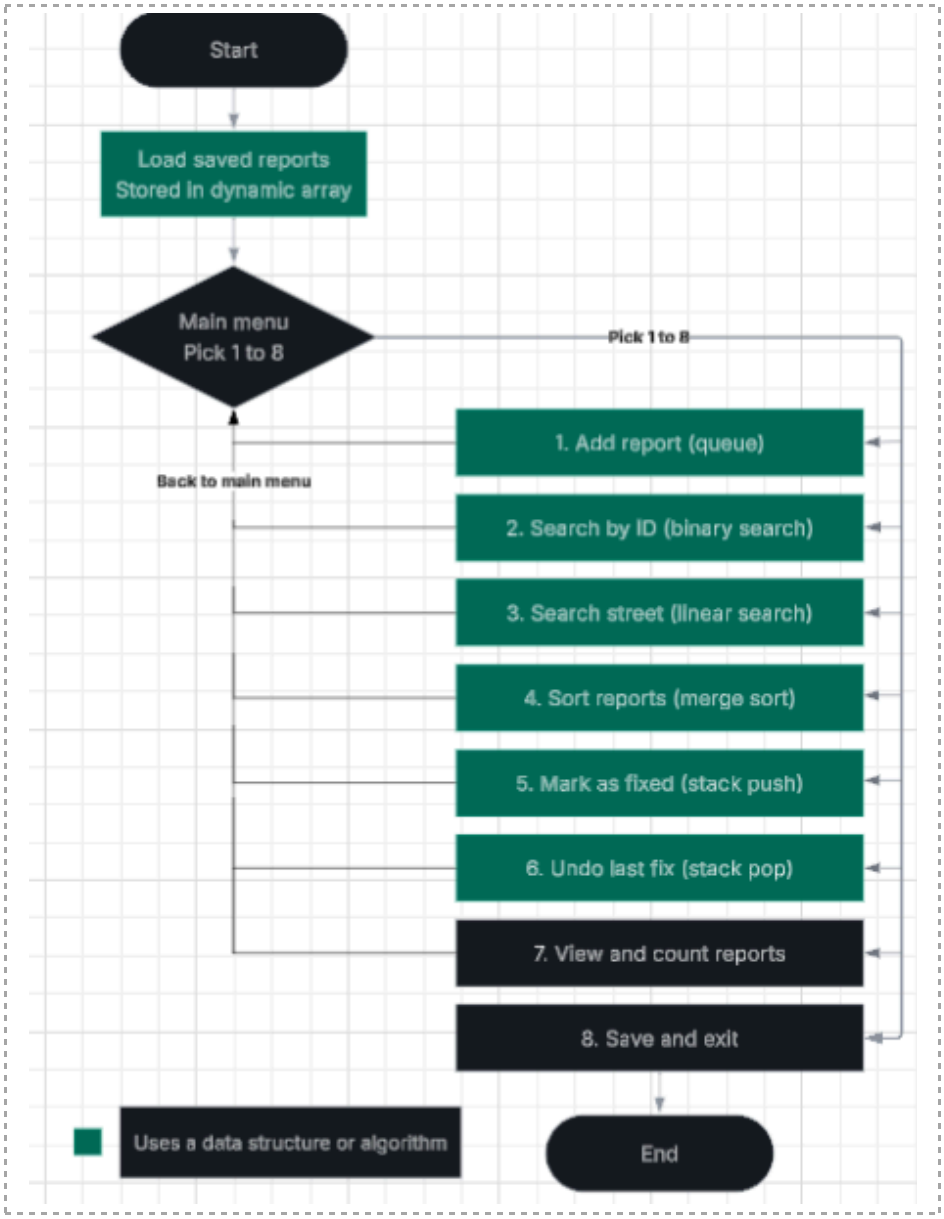


Figure 1. Flowchart of Barangay Flood Report and Clogged Drainage Tracking System

3.2 Flow Description

1. The program starts and loads the saved reports from the text file into a **dynamic array**.
2. The main menu is shown and the staff picks 1 to 8. If the input is invalid (a letter or a number outside 1 to 8), an error message is shown and the menu appears again.
3. **Add report (1):** The staff types the report details. The program adds the report to the dynamic array and puts its ID at the back of the queue with the status “Pending”.
4. **Search by ID (2):** The program uses binary search on the sorted list and shows the matching report, or a “not found” message.
5. **Search street (3):** The program uses linear search to check every report and shows all reports with that street name.
6. **Sort reports (4):** The program uses merge sort to arrange the reports by severity, highest flood level first, and shows the list.
7. **Mark as fixed (5):** The staff types the report ID. The status becomes “Fixed” and the report is pushed onto the stack.
8. **Undo last fix (6):** The program pops the top report from the stack and changes its status back to “Pending”.
9. **View and count reports (7):** The program shows the reports and counts them per street and per status.
10. After options 1 to 7, the program goes back to the main menu.
11. **Save and exit (8):** The program saves all reports to the text file and ends.

4. OUTPUT

Feature	Figure No.	Input Used
Feature 1: Record and monitor reports - main menu, add report (queue), mark as fixed (stack push), undo (stack pop), view and count	Figure 2	Program started (no input yet)
Feature 2: Find and prioritize reports - search by ID (binary search), search by street (linear search), sort by severity (merge sort)	Figure 3	1, Mabini St, level 3
Invalid input handling	Figure 4	abc, 9, blank street, severity 7, 0, x, non-existent ID, already-fixed report, empty undo, empty archive

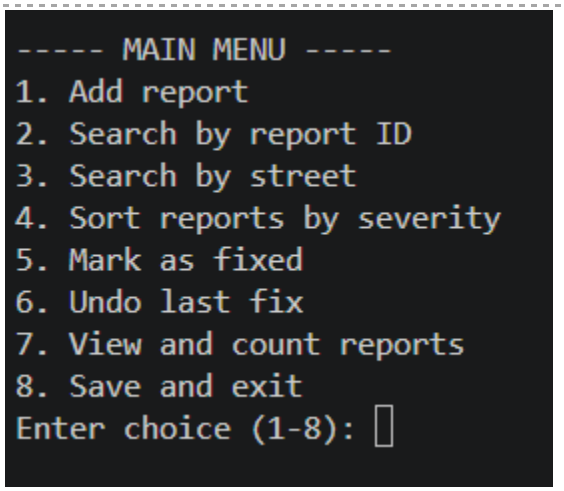


Figure 1. Main menu shown after the saved reports are loaded



Option 1- Add report: the new report gets the next ID (103) and goes to the back of the pending queue

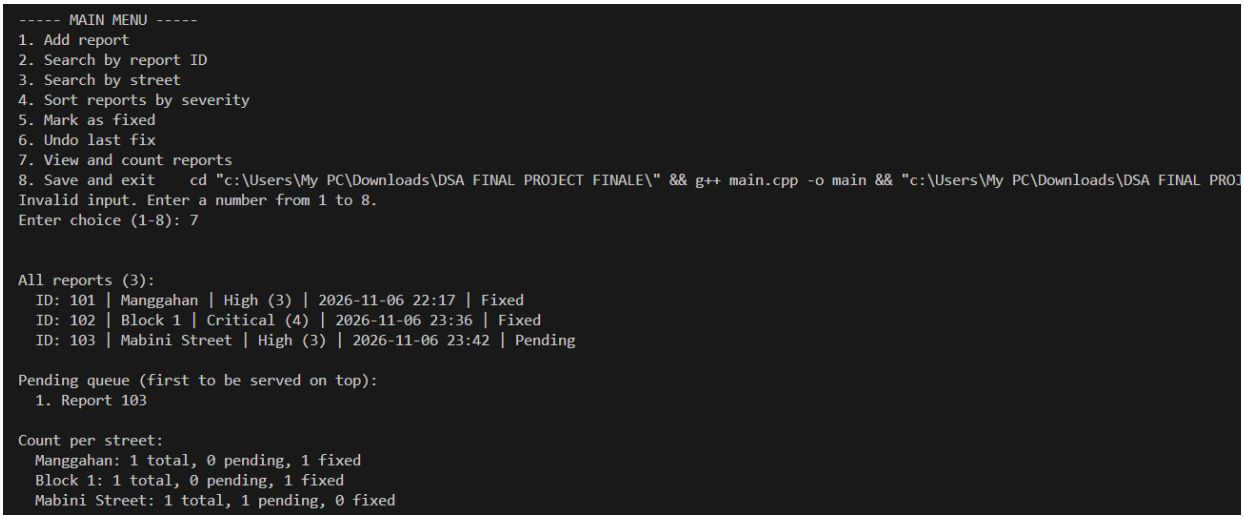
```
----- MAIN MENU -----
1. Add report
2. Search by report ID
3. Search by street
4. Sort reports by severity
5. Mark as fixed
6. Undo last fix
7. View and count reports
8. Save and exit
Enter choice (1-8): 5

Report ID to mark as fixed: 103
Report marked as fixed:
  ID: 103 | Mabini Street | High (3) | 2026-11-06 23:42 | Fixed
```

```
----- MAIN MENU -----
1. Add report
2. Search by report ID
3. Search by street
4. Sort reports by severity
5. Mark as fixed
6. Undo last fix
7. View and count reports
8. Save and exit
Enter choice (1-8): 6

Undo done. Report is pending again (back of the queue):
  ID: 103 | Mabini Street | High (3) | 2026-11-06 23:42 | Pending
```

Option 5 then Option 6 - Mark report 103 as fixed (stack push), then undo it (stack pop); 103 returns to Pending



Option 7 - View and count reports: full archive, pending queue (front on top), and count per street

Figure 2. Main menu, add report, mark as fixed, undo and view/count - queue, stack and dynamic array in action

```
----- MAIN MENU -----
1. Add report
2. Search by report ID
3. Search by street
4. Sort reports by severity
5. Mark as fixed
6. Undo last fix
7. View and count reports
8. Save and exit
Enter choice (1-8): 2

Report ID to search: 102
Report found:
  ID: 102 | Block 1 | Critical (4) | 2026-11-06 23:36 | Fixed

----- MAIN MENU -----
1. Add report
2. Search by report ID
3. Search by street
4. Sort reports by severity
5. Mark as fixed
6. Undo last fix
7. View and count reports
8. Save and exit
Enter choice (1-8): 2

Report ID to search: 1002
Report 1002 was not found.
```

Option 2 - Search by report ID (binary search): one report found, one ID not found

Figure 3. Searching and sorting output

5. PERFORMANCE ANALYSIS

5.1 Theoretical Analysis

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
Add report (enqueue ID)	Queue (linked list)	O(1)	O(1)	O(1)	O(1)
Remove from front (dequeue)	Queue (linked list)	O(1)	O(1)	O(1)	O(1)
Store report in archive (push_back)	Dynamic Array	O(1)	O(1) amortized	O(n) when it resizes	O(n)
Search by report ID	Binary Search	O(1)	O(log n)	O(log n)	O(1)
Search by street (all matches)	Linear Search	O(n)	O(n)	O(n)	O(n)
Sort by severity	Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Mark as fixed	Binary search + queue removal + stack push	O(n)	O(n)	O(n)	O(1)
Undo last fix	Stack pop + binary search + enqueue	O(1) if empty stack	O(log n)	O(log n)	O(1)
View and count per street	Nested loops over the dynamic array	O(n)	O(n x k)	O(n^2)	O(1)

Derivation:

Binary search by ID. The binarySearch algorithm performs a binary search by comparing the middle record in the array and eliminating half of the remaining portion of the search area in each pass of the loop. k passes at most n/2^k records which eventually goes to 1, so k = log2 n passes. 1,000 reports need about 10 passes and 100,000 need about 17. The best case happens when the first middle record is the one being searched, which will take O(1) time. There are only few variables used in the code, therefore the space is O(1).

Merge sort by severity. mergeSortRange breaks the range down into two halves until it contains only one record, which makes the recursion depth log2 n. The mergeParts function walks through all n records exactly once at each level, and so has T(n) = O(n log n). There is no early exit from the code and hence best case, average case and worst case are the same. It's using a temporary array of n records (temp), thus this space is O(n). Also sorts a copy of the archive, archive remains in report ID order for binary search.

Linear search by street. The while loop in linearSearchAll should perform the search for all reports of a street, so it must not terminate before n times, but also n times, and each iteration must lowercase the string of the street name of that report: The optimal running time is thus O(n) and not O(1).

Mark as fixed. The binary search time is O(log n), while the push onto stack costs O(1), but removeFromQueue takes time O(n) for counting the n IDs in the queue and O(n) for removing the ones from the queue and putting them back in again, except those whose ID is being removed. That is n dequeue operations and n-1 enqueue operations,

thus approximately 2n queue operations. When the report is at the front, a queue does allow access to the front and this is always O(n).

Dynamic array push_back. The capacity begins at 4 and doubles when it is full. The resizes which are O(n) are performed only when the size is 4, 8, 16, ... and the amount of copying performed for n additions is less than 4 + 8 + ... + n, which is less than 2n. Doing n additions, each one is O(1) on average

View and count per street. If it finds that the street was already counted in some earlier report in the code, it goes to the previous reports j < i and assumes that the street has been counted before; otherwise, if the street was not previously counted in some report, it scans over all n reports to count the street. If all the reports are located on a single street, the check can be done at once, and the work is O(n). O(n2) if all reports are on different streets, both loops will be executed.

5.2 Experimental Analysis

Setup Item	Details
Computer (CPU, RAM)	Ryzen 7 5700X, 16 GB RAM
Operating system	Windows 11
Compiler and settings	g++ 16.1.0 (MSYS2 project build, Rev5), flags: -std=c++17 -O2
Timing method	Std::chrono::steady_clock around the operation only; fast operations are repeated and divided
Input data	Randomly generated report (fixed seed, 50 street names, severity 1-4); same data for every algorithm
Repetitions	5 runs per input size; average reported

Input Size (n)	Binary Search Time (ms)	LinearSearch Time (ms)	MergeSort Time (ms)	Enqueue(n total) Time (ms)	Push(n total) Time (ms)	Pop(n total) Time (ms)	MarkFixed(1) Time (ms)
100	0.000019	0.025357	0.164300	0.002720	0.002480	0.001500	0.003519
1,000	0.000044	0.254697	2.224160	0.034820	0.029440	0.013080	0.035545
10,000	0.000111	2.541375	24.771580	0.319220	0.270900	0.135480	0.348980
100,000	0.000199	25.763344	308.861240	3.031960	3.003480	1.489620	3.629848

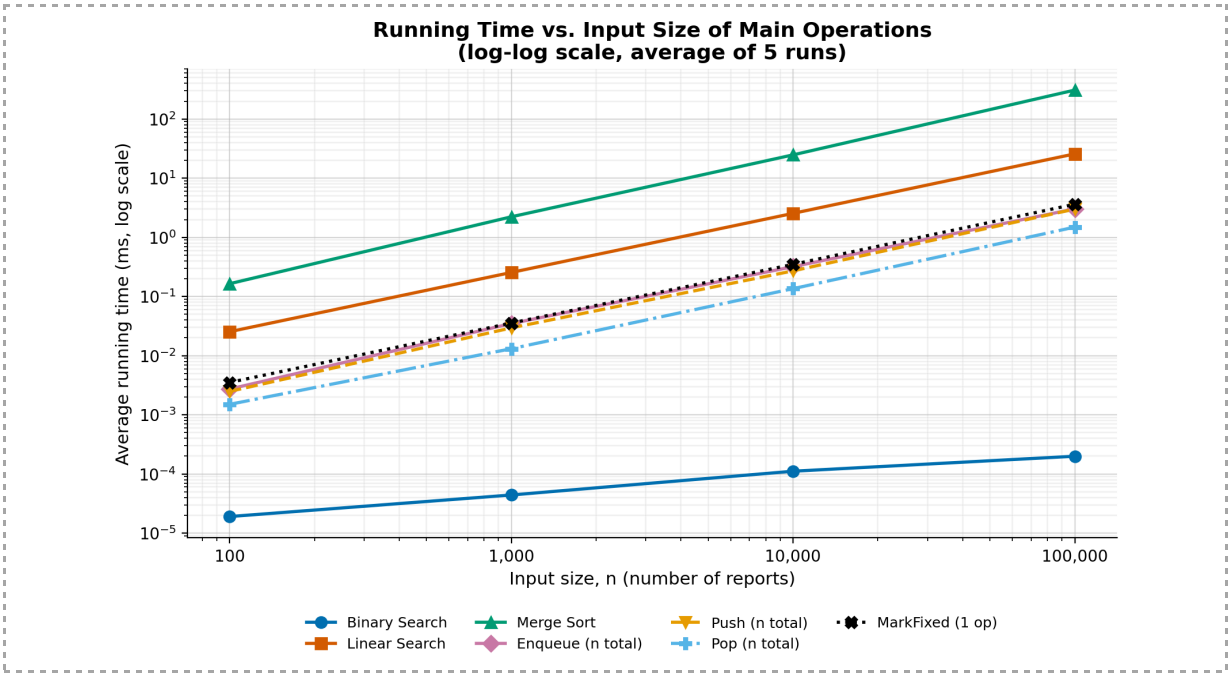


Figure 5. Running time vs. input size of main operations

5.3 Comparison of Theoretical and Experimental Results

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
Linear Search	O(n) growth about 10x per 10x increase in n	About 10x per step	Yes
Merge Sort	O (n log n) growth a bit more than 10x per step	About 11x to 13.5x per step	Yes
Binary Search	O (log n) does grows very slowly	It stayed tiny but it grew for about 10x overall	Mostly
Enqueue (n total)	O(n) it growth for total for n operations for about 10x	About 9x to 13x per its step	Yes
Push (n total)	O(n) total for n operations is all about 10x	About 9x to 12x per its step	Yes
Pop (n total)	O(n) total for n operations is all about 10x	About 9x to 11x per its step	Yes
MarkFixed	O(n) is about 10x	About 10x per step	Yes

Discussion:

Most of our results are the same as the result we got from Big-O.

The most obvious one is linear search. Every time n went up 10 times, the time also went up about 10 times (0.025 ms at n = 100 to 25.76 ms at n = 100,000). This is logical as it reviews each report individually.

merge sort increased at a rate of 11 to 13 times per step, a slightly larger increase than the 10. That's what O(n log n) should be, since it's O(n) with the additional log n component. It also is why the merge sort line on the graph is above all other lines.

Push, enqueue & pop both are O(1). However, we made a total of n operations, the overall time will increase approximately 10 times with each step, and it did. MarkFixed also increased by 10 times for each step, as for removeFromQueue() it costs O(n).

The only one that doesn't fit exactly is binary search. According to O(log n) the time should increase by at most 10% from 100 to 100000 and it increases by about 10X times. Despite the small times, however (0.00002 to 0.0002ms), the line is much flatter than the others. The difference we believe is due to the way we are caching and the accuracy of our timers, as a small array can fit in the CPU cache while a large one can't.

The small differences (such as the 12.8-fold growth of enqueue at the beginning) are likely due to the small amount of input, load on the system, and precision of the timer, as these operations only take a few microseconds.

Overall, the experiment agrees with our theoretical discussion in Section 5.1.

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The system follows the barangay's flood complaint process. Complaints are added to a queue. Stored in a dynamic array. Users can search for reports using the report ID or the street name. Reports can also be sorted by severity level. If a report was mistakenly marked as fixed it can be undone. This new system replaces the logbook and sticky-note method. It also fixes the problems that were, in those ways. Those problems were lost records, no ranking of reports and no way to track status.

Based on our trial and errors, the objectives were met. Reports are served on a come first-served basis. Both searches work correctly. Merge sort arranges the severe reports first. Running times were analyzed in theory. Confirmed through experiments.

The benchmarks shows that the measured times mostly matched the Big-O predictions. Linear search took 10 times longer for each tenfold increase in n. At n = 100 it was 0.026 ms and at n = 100,000 it reached 26.17 ms. Merge sort grew by 13 times per step which is a little more than linear. This fits the O(n log n) prediction. Binary search stayed very small going from 0.000018 ms, to 0.000203 ms. Total enqueue time grew in proportion to n.

6.2 Recommendations

- Make the removal of a report from the queue more efficient. Dequeueing a report as fixed takes O(n), since removeFromQueue() dequeues all the IDs and then enqueues them again. By implementing a doubly linked list that deletes the report, this will take O(1).
- Use a hash table to count the number of reports on each street. "View and count" function uses two nested loops, which takes $O(n^2)$ when there is only one report on each street. With the hash table, it will take O(n).

7. REFERENCES

Apacible, L. M. (2026, August 8). Habagat floods parts of Metro Manila, roads remain passable.

https://tribune.net.ph/2026/08/08/habagat-floods-parts-of-metro-manila-roads-remain-passable

Binary search algorithm and time complexity explained. (n.d.). freeCodeCamp. https://www.freecodecamp.org/news/binary-search-algorithm-and-time-complexity-explained/

Demaine, E., Ku, J., & Solomon, J. (2020). Lecture 3: Sorting. In 6.006 Introduction to algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec3/

Demaine, E., Ku, J., & Solomon, J. (2020). Lecture 5: Linear sorting. In 6.006 Introduction to algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec5/

APPENDICES

Appendix A. Source Code

A.1 main.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <ctime>
#include <cstdlib>
#include "FloodReport.h"
#include "DynamicArray.h"
#include "Queue.h"
#include "Stack.h"
#include "MergeSort.h"
#include "BinarySearch.h"
#include "LinearSearch.h"

const std::string FILE_NAME = "reports.txt";

// ---------- input helpers ----------
int readInt(const std::string& prompt, int low, int high) {
    while (true) {
        std::cout << prompt;
        std::string line;
        if (!std::getline(std::cin, line)) std::exit(0);

        std::istringstream iss(line);
        int value;
```

```
        char extra;
        if (iss >> value && !(iss >> extra) && value >= low && value <= high) {
            return value;
        }
        std::cout << "Invalid input. Enter a number from " << low << " to " << high << ".\n";
    }
}

std::string readText(const std::string& prompt) {
    while (true) {
        std::cout << prompt;
        std::string line;
        if (!std::getline(std::cin, line)) std::exit(0);

        for (size_t i = 0; i < line.size(); i++) {
            if (line[i] == '|') line[i] = ' ';   // '|' is used in the save file
        }
        size_t first = line.find_first_not_of(' ');
        if (first != std::string::npos) {
            return line.substr(first, line.find_last_not_of(' ') - first + 1);
        }
        std::cout << "Input cannot be empty.\n";
    }
}

std::string currentDateTime() {
    std::time_t now = std::time(nullptr);
    char buffer[20];
    std::strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M", std::localtime(&now));
    return std::string(buffer);
}

void printReport(const FloodReport& r) {
    std::cout << "  ID: " << r.id << " | " << r.street
              << " | " << severityName(r.severity) << " (" << r.severity << ")"
```

```
              << " | " << r.dateTime << " | " << r.status << "\n";
}

// ---------- comparisons / keys ----------
bool higherSeverityFirst(const FloodReport& a, const FloodReport& b) {
    return a.severity > b.severity;
}
int getId(const FloodReport& r) { return r.id; }
std::string getStreetLower(const FloodReport& r) { return toLowerStr(r.street); }

// ---------- the system ----------
class FloodSystem {
private:
    DynamicArray<FloodReport> archive;   // all reports, in order of report ID
    qNode<int>* front = nullptr;         // queue of pending report IDs
    qNode<int>* back = nullptr;
    sNode<int>* top = nullptr;           // stack of reports marked as fixed
    int nextId = 101;

    int findIndex(int id) {
        return binarySearch(archive.data(), archive.size(), id, getId);
    }

    // take the report out of the queue using only queue operations
    void removeFromQueue(int id) {
        int n = 0;
        for (qNode<int>* c = front; c != nullptr; c = c->next) n++;
        for (int i = 0; i < n; i++) {
            int x = 0;
            dequeue(&front, &back, x);
            if (x != id) enqueue(&front, &back, x);
        }
    }

public:
```

```
~FloodSystem() {
    clearQueue(&front, &back);
    clearStack(&top);
}

void load() {
    std::ifstream in(FILE_NAME);
    if (!in) return;
    std::string line;
    while (std::getline(in, line)) {
        std::istringstream iss(line);
        std::string idText, street, sevText, dateTime, status;
        if (!std::getline(iss, idText, '|') || !std::getline(iss, street, '|') ||
            !std::getline(iss, sevText, '|') || !std::getline(iss, dateTime, '|') ||
            !std::getline(iss, status)) continue;
        try {
            FloodReport r;
            r.id = std::stoi(idText);
            r.street = street;
            r.severity = std::stoi(sevText);
            r.dateTime = dateTime;
            r.status = status;
            archive.push_back(r);
            if (r.status == "Pending") enqueue(&front, &back, r.id);
            if (r.id >= nextId) nextId = r.id + 1;
        } catch (...) {}
    }
    std::cout << "Loaded " << archive.size() << " report(s) from " << FILE_NAME << ".\n";
}

void save() {
    std::ofstream out(FILE_NAME);
    for (int i = 0; i < archive.size(); i++) {
        FloodReport& r = archive[i];
        out << r.id << '|' << r.street << '|' << r.severity << '|'
```

```
                << r.dateTime << '|' << r.status << '\n';
        }
        std::cout << "Saved " << archive.size() << " report(s) to " << FILE_NAME << ".\n";
    }

    // 1. Add report (queue)
    void addReport() {
        FloodReport r;
        r.id = nextId++;
        r.street = readText("Street or block name: ");
        std::cout << "Severity: 1 = Low, 2 = Moderate, 3 = High, 4 = Critical\n";
        r.severity = readInt("Severity (1-4): ", 1, 4);
        r.dateTime = currentDateTime();
        r.status = "Pending";

        archive.push_back(r);
        enqueue(&front, &back, r.id);
        std::cout << "Report added to the queue:\n";
        printReport(r);
    }

    // 2. Search by report ID (binary search)
    void searchById() {
        if (archive.size() == 0) { std::cout << "No reports yet.\n"; return; }
        int id = readInt("Report ID to search: ", 1, 999999);
        int index = findIndex(id);
        if (index == -1) std::cout << "Report " << id << " was not found.\n";
        else { std::cout << "Report found:\n"; printReport(archive[index]); }
    }

    // 3. Search by street (linear search)
    void searchByStreet() {
        if (archive.size() == 0) { std::cout << "No reports yet.\n"; return; }
        std::string street = toLowerStr(readText("Street or block name to search: "));
        int* matches = new int[archive.size()];
```

```
        int found = linearSearchAll(archive.data(), archive.size(), street, getStreetLower,
matches);
        if (found == 0) {
            std::cout << "No reports found for that street.\n";
        } else {
            std::cout << found << " report(s) found:\n";
            for (int i = 0; i < found; i++) printReport(archive[matches[i]]);
        }
        delete[] matches;
    }

    // 4. Sort by severity (merge sort) - sorts a copy so the archive stays in ID order
    void sortReports() {
        int n = archive.size();
        if (n == 0) { std::cout << "No reports yet.\n"; return; }
        FloodReport* copy = new FloodReport[n];
        for (int i = 0; i < n; i++) copy[i] = archive[i];

        mergeSort(copy, n, higherSeverityFirst);

        std::cout << "Reports sorted by severity (most serious first):\n";
        for (int i = 0; i < n; i++) printReport(copy[i]);
        delete[] copy;
    }

    // 5. Mark as fixed (stack push)
    void markFixed() {
        if (archive.size() == 0) { std::cout << "No reports yet.\n"; return; }
        int id = readInt("Report ID to mark as fixed: ", 1, 999999);
        int index = findIndex(id);
        if (index == -1) { std::cout << "Report " << id << " was not found.\n"; return; }
        if (archive[index].status == "Fixed") { std::cout << "That report is already
fixed.\n"; return; }

        archive[index].status = "Fixed";
        removeFromQueue(id);
```

```
    push(&top, id);
    std::cout << "Report marked as fixed:\n";
    printReport(archive[index]);
}

// 6. Undo last fix (stack pop)
void undoFix() {
    int id;
    if (!pop(&top, id)) { std::cout << "Nothing to undo.\n"; return; }
    int index = findIndex(id);
    archive[index].status = "Pending";
    enqueue(&front, &back, id);   // goes back to the end of the queue
    std::cout << "Undo done. Report is pending again (back of the queue):\n";
    printReport(archive[index]);
}

// 7. View and count reports
void viewReports() {
    int n = archive.size();
    if (n == 0) { std::cout << "No reports yet.\n"; return; }

    std::cout << "\nAll reports (" << n << "):\n";
    for (int i = 0; i < n; i++) printReport(archive[i]);

    std::cout << "\nPending queue (first to be served on top):\n";
    if (isEmpty(front)) std::cout << "  (empty)\n";
    int pos = 1;
    for (qNode<int>* c = front; c != nullptr; c = c->next) {
        std::cout << "  " << pos++ << ". Report " << c->data << "\n";
    }

    std::cout << "\nCount per street:\n";
    for (int i = 0; i < n; i++) {
        std::string key = toLowerStr(archive[i].street);
        bool seen = false;
```

```
            for (int j = 0; j < i; j++) {
                if (toLowerStr(archive[j].street) == key) { seen = true; break; }
            }
            if (seen) continue;

            int total = 0, pending = 0, fixed = 0;
            for (int k = 0; k < n; k++) {
                if (toLowerStr(archive[k].street) == key) {
                    total++;
                    if (archive[k].status == "Pending") pending++; else fixed++;
                }
            }
            std::cout << "  " << archive[i].street << ": " << total << " total, "
                      << pending << " pending, " << fixed << " fixed\n";
        }
    }
};

int main() {
    std::cout << "=== Barangay Flood Report and Clogged Drainage Tracking System ===\n";
    FloodSystem system;
    system.load();

    int choice = 0;
    while (choice != 8) {
        std::cout << "\n----- MAIN MENU -----\n"
                  << "1. Add report\n"
                  << "2. Search by report ID\n"
                  << "3. Search by street\n"
                  << "4. Sort reports by severity\n"
                  << "5. Mark as fixed\n"
                  << "6. Undo last fix\n"
                  << "7. View and count reports\n"
                  << "8. Save and exit\n";
        choice = readInt("Enter choice (1-8): ", 1, 8);
```

```
        std::cout << "\n";


        if (choice == 1) system.addReport();
        else if (choice == 2) system.searchById();
        else if (choice == 3) system.searchByStreet();
        else if (choice == 4) system.sortReports();
        else if (choice == 5) system.markFixed();
        else if (choice == 6) system.undoFix();
        else if (choice == 7) system.viewReports();
        else system.save();
    }
    return 0;
}
```

A.2 Queue.h

```
#ifndef QUEUE_H
#define QUEUE_H
#include <iostream>


template <typename T>
class qNode {
public:
    T data;
    qNode* next;
};


template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
```

```
template <typename T>
bool isEmpty(qNode<T>* frontPtr) {
    return frontPtr == nullptr;
}

template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData) {
    qNode<T>* newNode = new_node(newData);

    if (*frontPtr == nullptr && *backPtr == nullptr) {
        *frontPtr = newNode;
        *backPtr = newNode;
    } else {
        (*backPtr)->next = newNode;
        *backPtr = newNode;
    }
}

template <typename T>
bool peek(qNode<T>* frontPtr, T& result) {
    if (frontPtr == nullptr) return false;
    result = frontPtr->data;
    return true;
}

template <typename T>
bool dequeue(qNode<T>** frontPtr, qNode<T>** backPtr, T& result) {
    if (*frontPtr == nullptr) return false;

    qNode<T>* deleteNode = *frontPtr;
    result = deleteNode->data;
    *frontPtr = (*frontPtr)->next;
    if (*frontPtr == nullptr) {
        *backPtr = nullptr;
    }
```

```
    delete deleteNode;
    return true;
}

template <typename T>
void clearQueue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    T temp;
    while (dequeue(frontPtr, backPtr, temp)) {}
}

#endif
```

A.3 Stack.h

```
#ifndef STACK_H
#define STACK_H

template <typename T>
class sNode {
public:
    T data;
    sNode* next;
};

template <typename T>
bool isEmpty(sNode<T>* topPtr) {
    return topPtr == nullptr;
}

// add on top - O(1)
template <typename T>
void push(sNode<T>** topPtr, T newData) {
    sNode<T>* newNode = new sNode<T>;
    newNode->data = newData;
    newNode->next = *topPtr;
```

```
    *topPtr = newNode;
}

template <typename T>
bool peek(sNode<T>* topPtr, T& result) {
    if (topPtr == nullptr) return false;
    result = topPtr->data;
    return true;
}

template <typename T>
bool pop(sNode<T>** topPtr, T& result) {
    if (*topPtr == nullptr) return false;

    sNode<T>* deleteNode = *topPtr;
    result = deleteNode->data;
    *topPtr = (*topPtr)->next;
    delete deleteNode;
    return true;
}

template <typename T>
void clearStack(sNode<T>** topPtr) {
    T temp;
    while (pop(topPtr, temp)) {}
}

#endif
```

A.4 Mergesort.h

```
// [Screenshot the full contents of main.cpp here]
#ifndef MERGE_SORT_H
#define MERGE_SORT_H

template <typename T, typename Compare>
```

```
void mergeParts(T arr[], T temp[], int left, int mid, int right, Compare goesBefore) {
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right) {
        if (goesBefore(arr[j], arr[i])) {
            temp[k++] = arr[j++];
        } else {
            temp[k++] = arr[i++];
        }
    }
    while (i <= mid)   temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int x = left; x <= right; x++) {
        arr[x] = temp[x];
    }
}

template <typename T, typename Compare>
void mergeSortRange(T arr[], T temp[], int left, int right, Compare goesBefore) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSortRange(arr, temp, left, mid, goesBefore);
    mergeSortRange(arr, temp, mid + 1, right, goesBefore);
    mergeParts(arr, temp, left, mid, right, goesBefore);
}

template <typename T, typename Compare>
void mergeSort(T arr[], int arrSize, Compare goesBefore) {
    if (arrSize < 2) return;
    T* temp = new T[arrSize];
    mergeSortRange(arr, temp, 0, arrSize - 1, goesBefore);
```

```
    delete[] temp;
}


#endif
```

A.5 Binarysearch.h

```
#ifndef BINARY_SEARCH_H
#define BINARY_SEARCH_H


template <typename T, typename K, typename GetKey>
int binarySearch(T data[], int arrSize, K findKey, GetKey getKey) {
    int low = 0;
    int high = arrSize - 1;


    while (low <= high) {
        int mid = low + (high - low) / 2;
        K midKey = getKey(data[mid]);


        if (midKey == findKey) {
            return mid;
        }
        if (midKey < findKey) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}


#endif
```

A.6 Linearsearch.h

```
#ifndef LINEAR_SEARCH_H
#define LINEAR_SEARCH_H
```

```
template <typename T, typename K, typename GetKey>
int linearSearchAll(T data[], int arrSize, K findKey, GetKey getKey, int matches[]) {
    int found = 0;
    int i = 0;

    while (i < arrSize) {
        if (getKey(data[i]) == findKey) {
            matches[found] = i;
            found++;
        }
        i++;
    }
    return found;
}

#endif
```

A.7 Dynamicarray.h

```
#ifndef DYNAMIC_ARRAY_H
#define DYNAMIC_ARRAY_H

template <typename T>
class DynamicArray {
private:
    T* items;
    int count;
    int capacity;

    void grow() {
        capacity = capacity * 2;
        T* bigger = new T[capacity];
        for (int i = 0; i < count; i++) {
            bigger[i] = items[i];
        }
```

```
        delete[] items;
        items = bigger;
    }

public:
    DynamicArray() {
        capacity = 4;
        count = 0;
        items = new T[capacity];
    }
    ~DynamicArray() { delete[] items; }
    DynamicArray(const DynamicArray&) = delete;
    DynamicArray& operator=(const DynamicArray&) = delete;

    void push_back(const T& value) {
        if (count == capacity) grow();
        items[count] = value;
        count++;
    }
    T& operator[](int index) { return items[index]; }
    int size() const { return count; }
    T* data() { return items; }
};

#endif
```

A.8 Floodreport.h

```
#ifndef FLOOD_REPORT_H
#define FLOOD_REPORT_H
#include <string>
#include <cctype>

struct FloodReport {
    int id = 0;
    std::string street;
```

```
    int severity = 1;
    std::string dateTime;
    std::string status = "Pending";
};

inline std::string severityName(int s) {
    if (s == 1) return "Low";
    if (s == 2) return "Moderate";
    if (s == 3) return "High";
    return "Critical";
}

inline std::string toLowerStr(std::string text) {
    for (size_t i = 0; i < text.size(); i++) {
        text[i] = std::tolower((unsigned char)text[i]);
    }
    return text;
}

#endif
```

Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	%
1	Aguila, Daniel	2512505	Documentation and System Developer	Prepared the project objectives, scope, and business process documentation. Assisted in developing the FloodReport structure and Dynamic Array implementation.	20 %

2	Acelajado, John Vince D.	2516965	System Developer	Worked on the system flowchart and program flow documentation. Developed the main program structure, input validation, and file handling.	20 %
3	Igloso, Denzen D.	1913789	Algorithm Developer	Prepared the explanations and applications of searching algorithms. Implemented Binary Search and Linear Search and helped verify their outputs.	20 %
4	Diez, Angela May	2511880	Algorithm and Performance Analyst	Prepared the sorting algorithm documentation and performance analysis. Implemented Merge Sort and assisted in analyzing the program's execution time.	20 %
5	Duenas,Ralph	2516015	System Developer and Tester	Prepared the system output documentation and testing procedures. Developed the Queue and Stack operations and assisted in testing the system's features.	20 %

Appendix C. GitHub Repository

Item	Details
Repository link	JohnAcelajado/DSA_FINAL_PROJECT_GROUP4
Branch / release submitted	main (latest commit c57f182, 12 commits total)

README contents	
How to compile	DSA_FINAL_PROJECT_GROUP4/DSA FINAL PROJECT FINALE at main · JohnAcelajado/DSA_FINAL_PROJECT_GROUP4

YOUTUBE LINK
youtube.com/watch?v=VP-Y6PkzFK4&fbclid=IwY2xjawVa17FleHRuA2FlbQIxMQBwZG9mA3NydGMGYXBwX2lkATAAAR8xSPPpUKktTaOwgUc0_rjC6YKpkW3YB_WILd9iR9T4lZqrqUgNxddoEuXEcg_aem_YoHDyw2wbsEqCUKI7X75iw

CANVA LINK
https://www.canva.com/design/DAHaJJpYWWQ/bCnyvqyChCZgpiO2s4sZkQ/edit

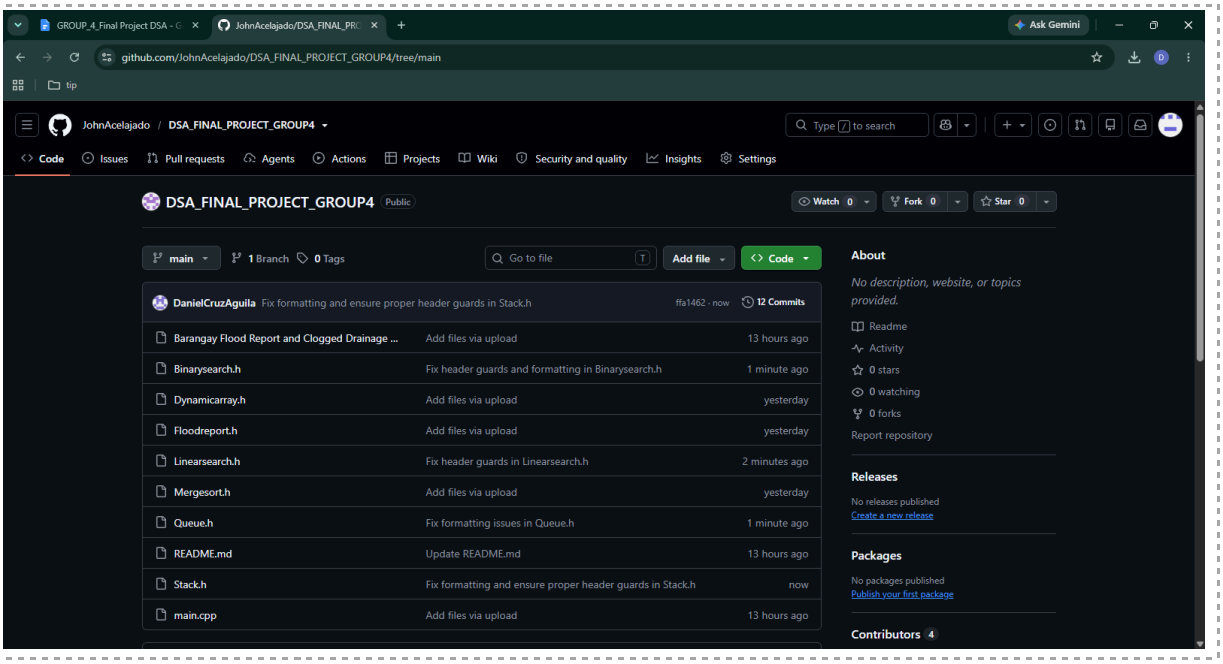


Figure 6. Screenshot of the repository page showing files and commit history

EVALUATION CRITERIA

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15
Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100